Data Engineering Curriculum

Day 36: The Medallion Architecture & Multi-Hop Pipelines

Day 36 Core Concept: The Medallion Architecture

Following our deep dive into Delta Lake, Day 36 focuses on structuring our Data Lakehouse using the **Medallion Architecture**. This is a data design pattern used to logically organize data in a Lakehouse, progressively improving the structure and quality of data as it flows through multiple layers.

- **Bronze Layer (Raw):** The landing zone. Data is ingested from external source systems in its rawest form (JSON, CSV, Parquet) and appended to Delta tables. History is preserved, and no data is dropped.
- **Silver Layer (Cleansed & Conformed):** The filtering zone. Data from the Bronze layer is matched, merged, conformed, and cleansed. We enforce schemas, drop duplicates, and standardize date/time formats here.
- **Gold Layer (Curated & Aggregated):** The consumption zone. Data is highly refined, aggregated, and modeled specifically for business reporting, BI dashboards, or Machine Learning feeds (e.g., Daily Active Users, Monthly Sales).

End-to-End Pipeline: ADF Orchestrating a Medallion Flow

> **Problem Statement:** A batch of raw web tracking events (`events.json`) drops daily into your Azure Data Lake Storage (ADLS).
>
> **Orchestration:** Use Azure Data Factory (ADF) with a Scheduled Trigger to execute a Databricks Notebook Activity.
> **Transformation:** In Databricks, build a multi-hop pipeline:
>
> 1. Read the raw JSON and save it as a **Bronze** Delta table.
> 2. Read from Bronze, drop invalid events (where `user_id` is null), standardize the timestamp, and save to a **Silver** Delta table.
> 3. Read from Silver, aggregate the data to calculate "Total Events per User per Day", and save it to a **Gold** Delta table for Power BI consumption.

The PySpark Implementation (Databricks Notebook)

This single notebook performs the multi-hop transformation, triggered seamlessly by ADF.

```
from pyspark.sql.functions import col, to_timestamp, date_format, count

# Define ADLS Storage Paths for Medallion Layers
source_json_path = "abfss://landing@.dfs.core.windows.net/events.json"

bronze_path = "abfss://lakehouse@.dfs.core.windows.net/bronze/events"
silver_path = "abfss://lakehouse@.dfs.core.windows.net/silver/events"
gold_path = "abfss://lakehouse@.dfs.core.windows.net/gold/daily_user_events"

# ------------------------------------------------------------
# STEP 1: BRONZE LAYER (Raw Ingestion)
# ------------------------------------------------------------
df_raw = spark.read.json(source_json_path)

# Save as Bronze Delta Table (Append mode to keep history of all raw loads)
df_raw.write.format("delta").mode("append").save(bronze_path)
print("Bronze layer load complete.")

# ------------------------------------------------------------
# STEP 2: SILVER LAYER (Cleansing & Formatting)
# ------------------------------------------------------------
df_bronze = spark.read.format("delta").load(bronze_path)

# Cleanse: Drop null users and cast timestamp
df_silver = df_bronze \
    .filter(col("user_id").isNotNull()) \
    .withColumn("event_timestamp", to_timestamp(col("raw_timestamp"))) \
    .withColumn("event_date", date_format(col("event_timestamp"), "yyyy-MM-dd"))

# Save as Silver Delta Table
df_silver.write.format("delta").mode("overwrite").save(silver_path)
print("Silver layer load complete.")

# ------------------------------------------------------------
# STEP 3: GOLD LAYER (Business Aggregations)
# ------------------------------------------------------------
# Aggregate: Count total events per user, per day
df_gold = df_silver.groupBy("event_date", "user_id") \
    .agg(count("event_id").alias("total_daily_events"))

# Save as Gold Delta Table
df_gold.write.format("delta").mode("overwrite").save(gold_path)
print("Gold layer load complete.")

# ------------------------------------------------------------
# OPTIONAL: Maintenance on Gold Table
# ------------------------------------------------------------
```

```
spark.sql(f"OPTIMIZE delta.`{gold_path}` ZORDER BY (event_date)")
```

Why Orchestrate this with ADF?

By using Azure Data Factory to trigger this notebook, you can easily parameterize the pipeline. For example, ADF can pass the exact filename (e.g., `events_2023_10_25.json`) into the Databricks notebook as a parameter, ensuring that Databricks only processes the newest data incrementally rather than the entire historical batch every day.